Reviewer Pack — Moment Matrix Spectral Gap and SOS Degree for Max-CUT

Entry 797c606cf901 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 03:38:02 UTC

Moment Matrix Spectral Gap and SOS Degree for Max-CUT

Entry ID: `797c606cf901` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 03:38:02 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

> For any n-variable Max-CUT instance, the smallest eigenvalue of its degree-d moment matrix satisfies λ_min ≥ Θ(1/√n) iff d ≥ Ω(log n). The spectral gap γ = λ_max - λ_min scales as γ = Θ(1/n) for d = Ω(log n).

Rationale (proposer's reasoning):

> Random matrix theory provides tools to analyze the spectral properties of moment matrices, which are central to SOS hierarchies. The spectral gap directly influences the convergence rate of SOS approximations, linking algebraic structure to algorithmic complexity.

Taxonomy category: `SOS_DEGREE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b6d10fcef4543813`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    d = math.ceil(math.log(n))
    instances_tested = 30

    def generate_3cnf_instance(n):
        clauses = []
        for _ in range(2 * n):
            literals = [random.choice([1, -1]) * random.randint(1, n) for _ in range(3)]
            clause = tuple(sorted(literals))
            if clause not in clauses:
                clauses.append(clause)
        return clauses

    def construct_moment_matrix(clauses, d):
        m = [[0] * (d + 1) for _ in range(d + 1)]
        for clause in clauses:
            for i in range(1, d + 1):
                for j in range(i, d + 1):
                    m[i][j] += sum(lit ** abs(j - k) for lit in clause)
                    m[j][i] = m[i][j]
        return m

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            # Find pivot
            max_row = i
            for j in range(i + 1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]

            # Eliminate below
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
```

```
            for k in range(i, n + 1):
                A[j][k] -= factor * A[i][k]

    # Back substitution
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = A[i][-1]
        for j in range(i + 1, n):
            x[i] -= A[i][j] * x[j]
        x[i] /= A[i][i]

    return x

def eigenvalues(A):
    n = len(A)
    if n == 1:
        return [A[0][0]]

    # Reduce to tridiagonal form
    T = [[0] * n for _ in range(n)]
    Q = [[0] * n for _ in range(n)]
    for i in range(n):
        Q[i][i] = 1

    for k in range(1, n):
        h = sum(A[i][k - 1] ** 2 for i in range(k, n)) ** 0.5
        if A[k - 1][k - 1] > 0:
            c = h / (A[k - 1][k - 1] + h)
            s = -A[k][k - 1] / (A[k - 1][k - 1] + h)
        else:
            c = -A[k][k - 1] / h
            s = h / A[k - 1][k - 1]

        T[k - 1][k - 1] = A[k - 1][k - 1] * c ** 2 + A[k][k - 1] * s ** 2 - 2 * A[k - 1][k] *
        T[k][k] = A[k][k] * c ** 2 + A[k - 1][k] * s ** 2 + 2 * A[k - 1][k] * c * s
        T[k - 1][k] = T[k][k - 1] = 0

        for i in range(k):
            T[i][k - 1] = T[k - 1][i] = 0

        for j in range(n):
            Q[j][k - 1] = Q[k - 1][j] = 0
            Q[j][k - 1] = c * Q[j][k - 1] + s * Q[j][k]
            Q[j][k] = -s * Q[j][k - 1] + c * Q[j][k]

    # Solve for eigenvalues of tridiagonal matrix
    def solve_tridiag(a, b, c, d):
        n = len(b)
        alpha = [0] * n
        beta = [0] * n
        y = [0] * n

        alpha[1] = -c[0] / b[0]
        beta[1] = d[0] / b[0]

        for i in range(2, n):
```

```
            alpha[i] = -c[i - 1] / (b[i - 1] + a[i - 1] * alpha[i - 1])
            beta[i] = (d[i - 1] - a[i - 1] * beta[i - 1]) / (b[i - 1] + a[i - 1] * alpha[i - 1
        y[n - 1] = (d[n - 1] - a[n - 2] * beta[n - 2]) / (b[n - 1] + a[n - 2] * alpha[n - 2])
        for i in range(n - 2, -1, -1):
            y[i] = alpha[i] * y[i + 1] + beta[i]

        return y

    def eigenvalues_tridiag(a, b, c):
        n = len(b)
        if n == 1:
            return [b[0]]

        a_prime = [a[0]] + [0] * (n - 2) + [c[-1]]
        b_prime = [b[0], b[1], b[n - 2], b[n - 1]]
        c_prime = [0] * (n - 2) + [c[0]]

        ev_left = eigenvalues_tridiag(a_prime, b_prime[:n-1], c_prime)
        ev_right = eigenvalues_tridiag(a_prime, b_prime[1:], c_prime)

        return sorted(ev_left + ev_right)

    T_flat = [T[i][j] for i in range(n) for j in range(i, n)]
    a = [0] * (n - 2)
    b = [T_flat[n - 1]] + [T_flat[i] for i in range(n - 2, 0, -1)] + [T_flat[0]]
    c = [0] * (n - 2)

    ev = eigenvalues_tridiag(a, b, c)
    return ev

def spectral_gap(eigenvalues):
    lambda_max = max(eigenvalues)
    lambda_min = min(eigenvalues)
    return lambda_max - lambda_min

total_lambda_min = 0
total_gamma = 0

for _ in range(instances_tested):
    instance = generate_3cnf_instance(n)
    moment_matrix = construct_moment_matrix(instance, d)
    eigenvals = gaussian_elimination(moment_matrix)
    lambda_min = min(eigenvals)
    gamma = spectral_gap(eigenvals)

    total_lambda_min += lambda_min
    total_gamma += gamma

mean_lambda_min = total_lambda_min / instances_tested
mean_gamma = total_gamma / instances_tested

conjecture_holds = mean_lambda_min >= 1 / math.sqrt(n) and mean_gamma >= 1 / n
counterexample = "" if conjecture_holds else "lambda_min < 1/√n or gamma < 1/n"

return {
```

```
        "metric_name": "spectral_gap",
        "metric_value": mean_gamma,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1d82cae.py", line 180, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1d82cae.py", line 153, in run_trial
    moment_matrix = construct_moment_matrix(instance, d)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1d82cae.py", line 39, in construct_mom
    m[i][j] += sum(lit ** abs(j - k) for lit in clause)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1d82cae.py", line 39, in <genexpr>
    m[i][j] += sum(lit ** abs(j - k) for lit in clause)
                                  ^
NameError: name 'k' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to undefined variable 'k' in the moment matrix construction, preventing data collection. | next: Debug the NameError in the genexpr by checking variable scoping for 'k' in the clause summation

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	99329
2	preregistration	ollama_remote	qwen3:8b	0	0	31612
3	novelty	ollama_remote	qwen3:8b	0	0	27302
4	novelty	ollama_remote	qwen3:8b	0	0	21033
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14809
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11693
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11083
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25424
9	judge	ollama_remote	qwen3:8b	0	0	21558

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 263843 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/797c606cf901.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/797c606cf901.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/797c606cf901.tar.gz` (if generated)